

A machine learning approach to bioinformatics

9

9.1 Introduction to machine learning?

A machine learning algorithm is a statistical computation method used in software to detect hidden patterns that are not obvious in a dataset and make reliable statistical predictions of similar new data. Machine learning techniques attempt to find a pattern in a particular dataset; using these learned patterns, a similar pattern in a new dataset is identified. Machine learning processes are somewhat close to statistical modeling and data collection. They look into the data to find trends and accordingly learn the pattern or parameters. We usually know about machine learning through social media connection suggestions, online shopping product recommendations, spam filters in our email inboxes etc. In the last 30 years, there has been a surge in the use of machine learning techniques to solve biological problems, with successes in gene prediction, protein function prediction, cell image recognition, pathway analysis, protein structure prediction, drug molecule and toxicity prediction, and so on. To solve a problem using a machine learning algorithm, we require data about the concerned problem, which consists of features or characteristics of problems. For example, the concept of automatic annotating proteins' function has become more prevalent in recent years because more and more proteins are being discovered and identified. In an attempt to comprehend the molecular mechanism of biology, it is important to assign the function to large-scale proteins. But only a tiny percentage of the more than 179 million UniProtKB proteins have experimentally supported gene ontology (GO) annotations. Here, machine learning can be applied as we have annotated proteins fed as examples to an ML algorithm. The ML algorithm can learn patterns from examples and help us predict newly discovered proteins' functions. Features are used to show examples of an ML algorithm. For the present case, features may be protein sequence, Position-Specific Scoring Matrix (PSSM), representing conservation and homology, protein–protein interaction (PPI) network data, motifs, domains etc. In simple terms, features are the information about a particular example under observation (Larrañaga et al., 2006).

Choosing descriptive, discriminating, and autonomous features is an essential step in training efficient machine learning algorithms. After feature selection, the next step is to identify the problem's category, based on which we choose an ML algorithm. We will discuss the different types of machine learning algorithms in

the subsequent sections. Then we train the models and evaluate their performance. According to recent machine learning footprints in life science, it seems to be helping researchers to address a variety of tiresome issues to gain greater understanding and opportunities for a prosperous future (Greener et al., 2021).

9.2 Types of machine learning systems

Machine learning has evolved from a science fiction fantasy to a widely used method in our community. It has such a significant impact on the outcome that incorporating machine learning algorithms into the process has become a deciding factor between success and failure. These algorithms are challenging to apply, and considerable effort has been expended in this area to get it to where it is today. It is important to comprehend what we want our algorithms to accomplish and the benefits they provide. Certain problems, such as predicting temperature, pH, or pressure, require a continuous numerical value as an output referred to as regression problems. In comparison, some questions require categorization, such as whether they are positive or negative or warm or cold. Additionally, there are instances where we expect machine learning models to classify examples based on the similarity and dissimilarity of their features. Machine learning models are generally classified into three categories based on their outcomes: supervised learning, unsupervised learning, and reinforcement learning.

9.2.1 Supervised learning

The most frequently used method of machine learning is supervised learning. It's the simplest concept to understand and put into practice. It's similar to using flash cards to teach a child something. We can feed a supervised algorithm data sequentially in the form of example-label pairs, allowing the algorithm to evaluate the label for each example and provide feedback on whether the label was estimated correctly. Finally, the algorithm will discover how to precisely interpret the relationship between instances and labels. Once thoroughly trained, a supervised learning algorithm will be capable of observing a new sample that has never been seen before and evaluating a suitable label for it. As a result, supervised learning is frequently referred to as task-based. It focuses on a single problem, training the algorithm with ample of examples until the task can be completed successfully. Additionally, supervision learning can be classified as regression or classification. Classification is generally used to predict a label, whereas regression is usually used to predict a quantity.

Regression is a technique for developing models that predict continuous values based on their input variables. In regression problems, the mathematical mapping function (f) from the input variable (x) to the output variable (y) is determined. Assume we have a dataset containing the mature height, weight, diet, and gender of our parents. Because height or weight is a continuous variable, training a supervised algorithm on these features to predict the child's height or weight is referred to as a

regression problem. Classification, on the other hand, is a technique for identifying a model that divides input data into a number of distinct classes or labels. In light of the aforementioned data set of height and weight, the classification task would be to use the adult height, weight, diet, and gender of the parents to predict the child's height as "Above Average" or "Below Average."

9.2.2 The below are the most commonly used supervised algorithms

9.2.2.1 Linear regression

Linear regression is a supervised learning algorithm used for the prediction of continuous variables like sales, age, product price, salary and future weather etc. Linear regression is represented by a linear equation in which one variable is independent (x), and the second one is the dependent variable, or the output (y).

If the input and output variables are linearly related, a line could be drawn with the data sets to depict the relationship between the variables, as shown in [Fig. 9.1](#). The equation for a basic linear regression model with a single independent variable x and a single dependent variable y will be:

$$y = mx + b$$

It is the equation of the line, where " x " is the input variable, y is the output variable, " m " is the slope, and " b " is the y -intercept. Linear regression aims to find the best values for " m " and " b " based on " x " and " y " data, so that the average difference

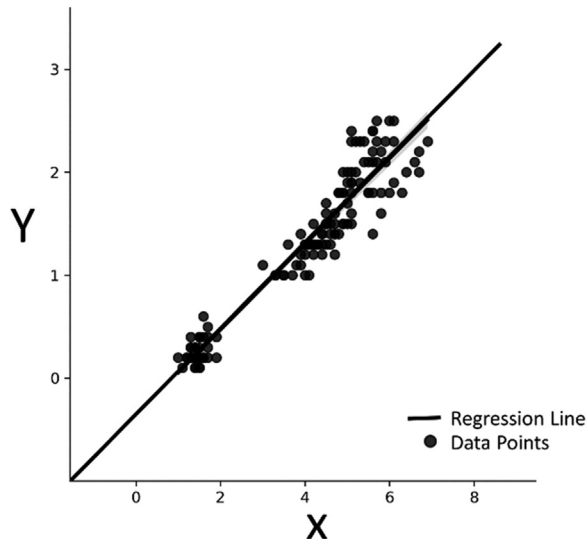


FIG. 9.1

Linear regression: Data points and the estimated best-fit regression line.

between the data points and the line is as small as possible. The difference between the data points and the line is referred to as an error of the model or the line, and each data point has its own error. The value obtained as these errors are squared and added is known as sum squared error, and the average of this sum is known as mean square error. The total error between all data points and a line is referred to as the mean square error, which states the model's efficiency.

Since “m” and “b” may have a wide range of values, regression iteratively finds the best line by fitting several lines and selecting the one with the lowest mean squared error (Fig. 9.2B). If there's more than two variables in a problem, the same approach is used; this is multivariate linear regression.

When there are more than two variables or more dimensions in the data, the regression line is a plane (in three dimensions) or hyper-plane (in more than three dimensions), and the equation is:

$$y = m_1x_1 + m_1x_1 + m_2x_2 + m_3x_3 + \dots \dots m_nx_n + b$$

Here, the dependent variable is y, and the independent variables or features are x. The coefficients of the features m_1 , m_2 , m_3 , m_4 , and m_n provide an understanding of each features's contribution in the measurement of “y”. Finally, the letter “b” is a bias term. We've seen how linear regression can be used to model a continuous variable, but the coefficients can often provide valuable knowledge about the significance or function of each feature in the estimation of y. We may also deduce approximate linear equations for problems, giving us an approximate mathematical model of the problem (Altman and Krzywinski, 2015).

9.2.3 Logistic regression

Classification problems involve recognizing spam or ham messages, deciding if a tumor is benign or malignant, determining whether or not a person has a disease,

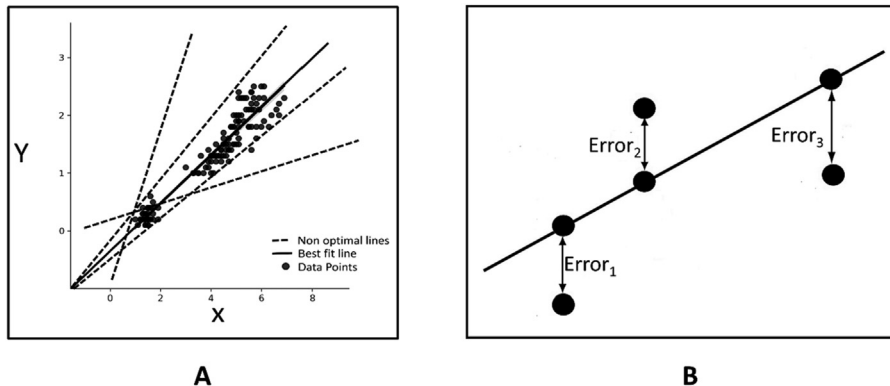


FIG. 9.2

(A) Best fit line among the various non-optimal lines and (B) Distance (errors) between the data points and the line.

and so on. These are regarded as binary classifications since there are just two classes. In general, two classes are indicated by the letters “0” and “1.” For example, a benign tumor is can be represented as “0,” whereas a malignant tumor can be represented as “1.” As a consequence, the model has to predict the output values of “0” or “1.” We cannot use linear regression equations since the output varies from $-(ve)$ infinity to $+(ve)$ infinity (Lever et al., 2016). So, for classification, we have a sigmoid function, which has an “S” shaped curve, as seen in Fig. 9.3.

9.2.4 K-nearest neighbor

The KNN algorithm can be best illustrated by the expression “A man is known by the company he keeps.” When a new disease is discovered, the doctor will recall previous patients who had the same symptoms. As a result, symptoms can be thought of as features, and labels as diagnostics or treatments for the disease (Parry et al., 2010). In KNN, the sorted training data set serves as a model representation, and no additional learning is required. Fig. 9.4 depicts the data points for two groups, blue and green, respectively. If we use $K = 3$ to create a new prediction event, the model will look for the three closest data points before assigning the new instance to the majority class with the highest probability. The diagram shows that the K value is an important factor in determining the model’s behavior. Low K values reduce model accuracy because noise has a greater impact on it, whereas high K values make the KNN algorithm computationally intensive. As a result, for greater accuracy, an optimal value of k is chosen. It is critical to select the appropriate k value to balance accuracy and computing power needs.

Various distance measures are used to determine which of the nearest neighbors is the closest. The most commonly used distance calculation is the Euclidian

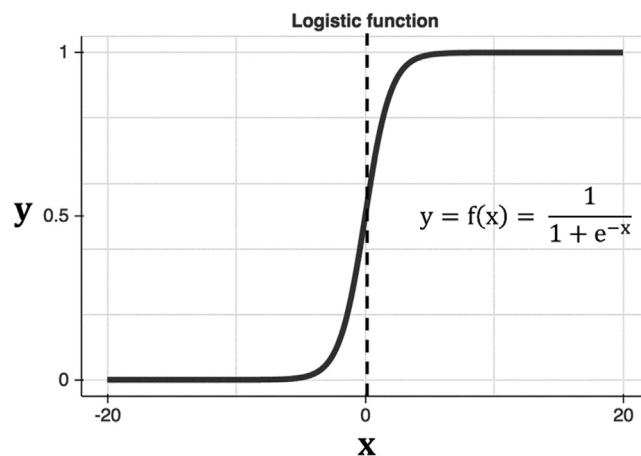


FIG. 9.3

The logistic or sigmoid function.

K Nearest Neighbor

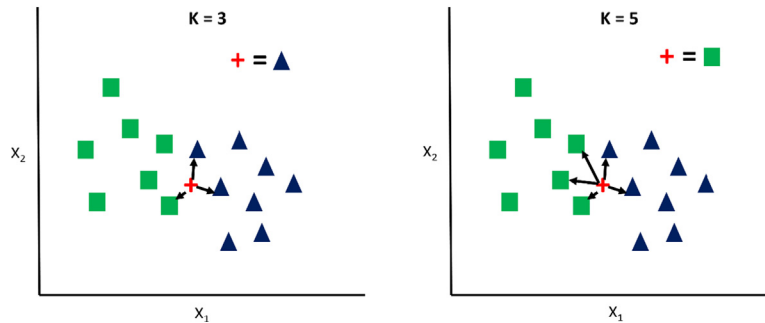


FIG. 9.4

Sinificance of “K” in K-nearest neighbor algorithm.

distance. The Hamming distance is also used, which is the sum of the absolute differences between unit vectors. The dimensionality issue arises as the number of features increases. The problem in a high-dimensional space is that all points are far away from one another or their neighbors (Fig. 9.4).

Several advantages of using KNNs include the following: The algorithm is straightforward and simple to train. Due to its adaptability, the KNN can be used for both classification and regression. KNN calculates the average value of the k nearest data points in regression; it is more consistent and reliable when dealing with a large training set. A few drawbacks of KNNs include the algorithm’s requirement for an exponential increase in computing power as the value of k increases. Its computational cost for predicting new data is quite high. When it comes to categorical features, KNN fails miserably.

9.2.5 Decision trees

The decision tree uses a tree-like hierarchical decision-making model to learn basic decision-making rules and predict the target variable’s value. It is a general technique that has applications in a number of real-world contexts, including civil engineering, law and business, and is also widely used in machine learning. Decision trees are used to accomplish specific goals, such as a flow chart or a series of decisions (Kotsiantis, 2011). Fig. 9.5 shows the standard structure and terminology of the decision tree. Each node is a test or condition, and the branches are the outcomes of such tests or conditions (Fig. 9.5).

- **Root Node:** This is where the entire population or sample is split into two or more segments.
- **Branches:** Branches are the result of node choices.

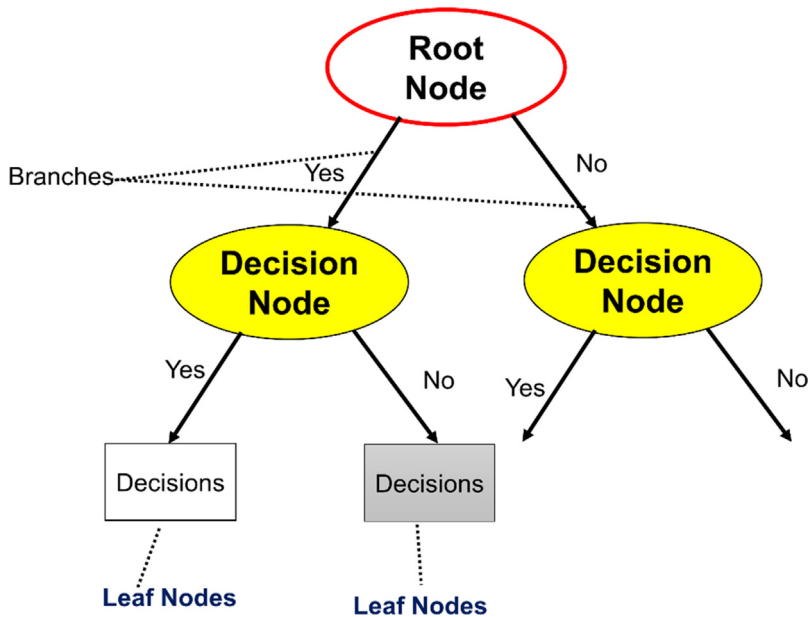


FIG. 9.5

Standard structure and terminology of the decision tree.

- **Decision Node:** When a sub-node is divided into additional sub-nodes, the decision node is formed.
- **Leaf Node:** Leaf nodes, also known as terminal nodes, are the final outcomes of all chain decisions.

Splitting is the method of separating a node into a number of sub-nodes depending on the test condition. Decision trees establish test conditions based on the purity or impurity gain from the decision in the lower subsets. The depth of the decision tree is increased with the number of decision nodes lead to the over-fitting of the model. As a result, we must restrict the creation of decision nodes or boundaries in real-world data while at the same time tolerating minor impurities in leaf nodes. The technique of deciding the depth of three is called tree pruning.

The following are some of the advantages of decision trees over other supervised learning algorithms:

1. Visually depicting and easy to understand
2. There is very little or no data processing needed.
3. It can be used for numerical as well as categorical results.

Decision trees, like every other machine learning algorithm, have their own set of drawbacks, including:

1. Excessive parameter tuning is required to avoid overfitting. Methods such as pruning and the determination of the minimum number of samples are used to solve the problem of overfitting.
2. The resulting tree may not be a globally optimal decision tree, because most decisions are made at individual nodes.
3. Most of the time a single tree is not enough to yield successful results.

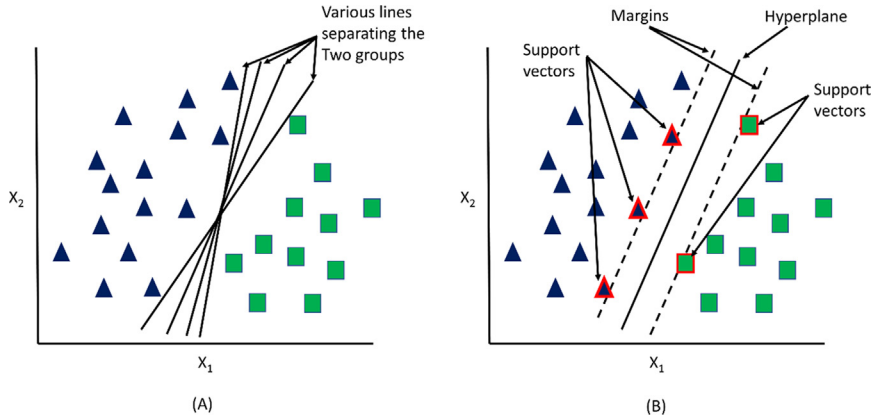
To resolve these drawbacks of decision trees, multiple decision trees' ability is used. A random forest is a collection of decision trees, as the name implies. A random forest classifier trains multiple decision trees on small samples of the training data points. These small samples are selected on a random basis with substitutes, resulting in a large number of decision trees. Bootstrap sampling is a tool for the selection of small samples. In order to produce the final result, the random forest algorithm integrates the contribution of individual decision trees. Since it is a set of multiple decision trees, the model is also known as the ensemble type model, where two or more models are used for deducing results. The result of the forecast depends on the overall observation of all trees, i.e., the majority vote. Random forests are much more reliable and resilient than single tree species. Random forests are more accurate than decision trees for two major reasons:

1. The random forests, unlike the decision trees, are not pruned, so the characteristic space is divided into smaller and smaller areas.
2. Each random forest tree learns from a random array, and each node divides on the basis of a random set of characteristics resulting in tree diversity.

9.2.6 Support vector machines

The support vector machine are generally used as supervised learning algorithm in machine learning. In addition to classification they can also be used in regression problems. In this section we will discuss the application of SVMs for the classification task. Support vector machines are linear classifiers, because they make linear decision boundaries. It aims to find a hyperplane or decision boundary that is the best fit and separates n -dimensional space into separate classes or groups (Wang and Lin, 2014). When we place the new instance to a trained SVM, it classifies the new data point into one of the categories. The optimal decision boundary is known as a hyperplane. The hyperplane is a flat decision boundary with dimensions $N-1$ for an N -dimensional dataset. For 2D data, visually, it will be a line, and for 3D data, it will be a plane separating two groups. For applying support vector machines, the data should be linearly classifiable, which means a line or plane should exist form where the groups or classes can be separated. The objective of SVM is to find that line of the hyperplane.

In Fig. 9.6A we can see that the dataset is linearly separable; however, there can be many lines that can separate the data into different classes. Support Vector Machine selects the optimal points which help in creating the hyperplane. The optimal points are the data points that are very similar but fall in different classes. SVM finds

**FIG. 9.6**

Algorithm behind support vector machines.

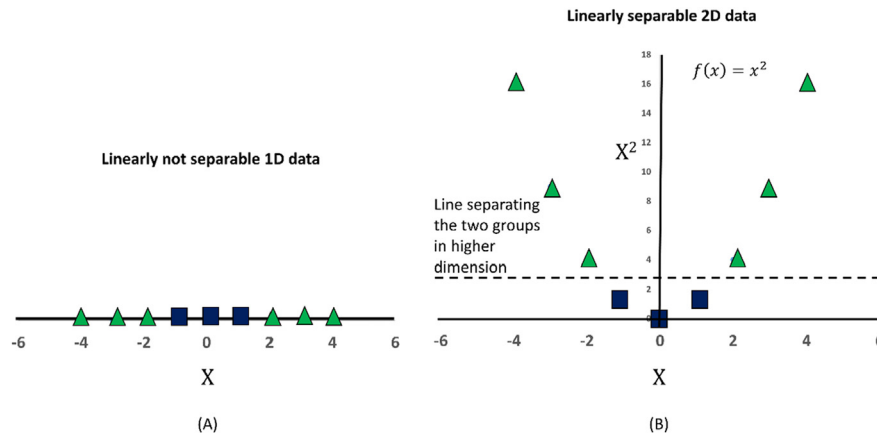
such points in the space and tries to fit a line or plane, which has a maximum distance from those datapoints. These points are called support vectors, as they help in finding the decision boundary, and the algorithm is called a support vector machine.

SVM optimizes the margin of the classifier by using these support vectors, and any change in these support vectors would change the hyperplane location. An example dataset is shown in Fig. 9.6. Here, the margins are dividing the points neatly, and this margin is called hard margin. With real-world data, we use a soft margin, which allows the SVM to misclassify a few points, while, maximizing the margin and minimizing the overall error. It is up to the user to decide the degree of tolerance or the softness of the margin. A sklearn's support vector machine class is controlled using a parameter called "C". For the lower value of "C" the model will have less tolerance for misclassification, and for the high value of "C" it will have more tolerance for misclassification.

9.2.6.1 Kernel trick

We have discussed that SVMs can only classify linearly separable data, but datasets in the real-world may not always be linearly separable. Although up to some extent, using a soft margin can help us to classify data linearly, even then, the data must have a linear discriminatory boundary. For datasets that do not have any single linear boundary, the kernel trick is applied, which aims to map the low dimensional data to a higher dimension where it is linearly separable. Let us understand it using a non linearly separable one dimensional data.

Fig. 9.7A is a one-dimensional dataset that has two classes, green and blue. There is no way a point can differentiate the two classes. One dimensional dataset is a line only, and the hyperplane will be a point here, as a point has zero dimension. The goal here is to add an extra feature to the data so that we can differentiate the two classes using a line in two-dimensional space. Here the new feature is the square of the data

**FIG. 9.7**

Mapping a non linearly separable one-dimensional data to high dimension to classify it linearly.

points. Therefore, in Fig. 9.7B, we can see that introduction of this new feature made the data points linearly separable into the two different classes. Therefore, the assumption here is that, if an N-dimensional data is not linearly separable, then there is a dimension greater than N where the datapoints are linearly separable. Kernel functions are the general functions which generate new features to map these data point to higher dimensions.

Radial basis function (rbf) is one of the common kernel functions used to perform kernel trick. “rbf” is used to create new higher dimensional features by calculating the distance among all other data points to a certain point. The mathematical representation of radial basis function is

$$k(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

where x_i and x_j are two instances and γ (gamma) controls the effect of new features on the decision boundary. Just as “C”, the regionalization parameter, gamma also needs to be tuned for optimal performance of the SVMs.

9.2.7 Neural networks

We will now address another commonly used supervised learning algorithm, Artificial Neural Networks, or ANNs, or Neural Nets, in this section. They were designed to mimic the neural networks or neurons that comprise the human brain, as the name implies. They are primarily used for statistical analysis and modeling of gathered data; their function is viewed as a complement to conventional nonlinear regression analysis models (Lecun et al., 2015). They are frequently used to solve problems that can be expressed as regression or classifications. With more than 6 decades of research behind them, neural networks have found applications in a wide variety of fields, including speech and image recognition and classification, text recognition, medical diagnosis, and fraud detection.

9.2.8 Neural networks architecture

The basic structure of neural networks is shown in Fig. 9.8. It is separated into three different layers: input, hidden, and output. The input layer is the first layer and it contains the input features or characteristics. The middle layer is the hidden layer, the word 'hidden' referring to the mathematical calculation processes that are not accessible and are often referred to as the black box. Diverse networks are classified according to the number of hidden layers they comprise. The ANN with many hidden layers is referred to as a deep neural network. The final layer is the output layer, which contains the network's output data.

With a large number of weights and activation functions, neural networks can accommodate both linear and nonlinear datasets. These characteristics of neural networks also make them prone to overfitting the training data. As a consequence, they require rigorous parameter tuning to achieve the best results. We will discuss more about overfitting toward the end of this chapter.

9.2.9 Convolutional neural network

To train an artificial neural network on image data, we must extract all the pixels from the image and pass each pixel as a variable to the neural network. Although individual pixels of an image provide information about it, images become more

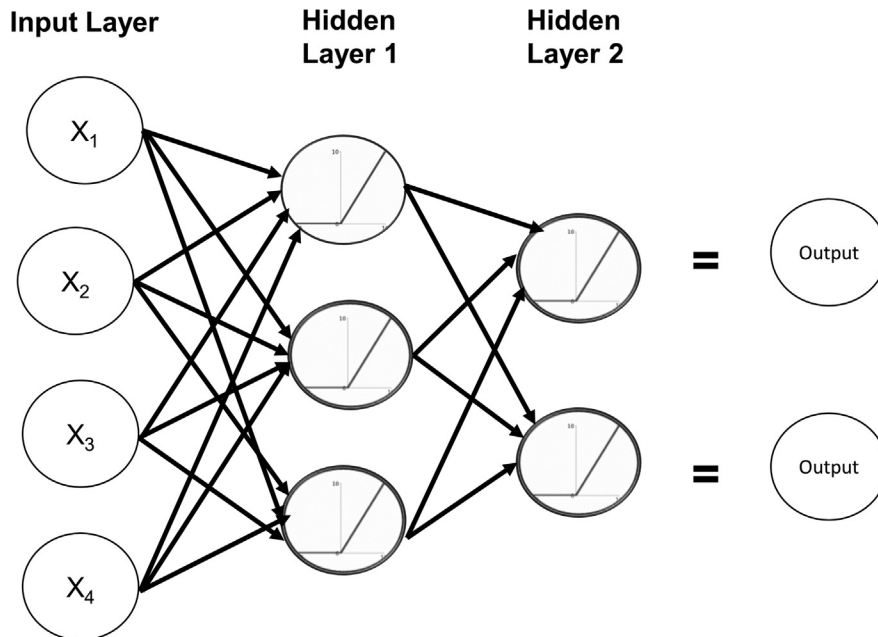


FIG. 9.8

Basic structure of neural networks.

recognizable when local pixels are combined. Convolutional neural networks (CNNs) are well-known models of deep learning for image or spatial data. CNN can be used to learn local spatial features, while ANN can be used to learn individual features. CNN learns through the use of a small window or filter. The picture is segmented into small sections, and several filters are used to learn various local features such as edges and textures. Thus, these individual image pixel features are combined to form higher-order features such as the boundaries and depths etc.

These filters are composed of convolutional matrices of numbers that can be trained or modified. With properly trained filters, CNN detects and extracts the appropriate features from image data (Fig. 9.9). Filters are trained using the back-propagation method with each iteration. By definition, convolution is a mathematical operation on two objects that results in the transformation of one object when it interacts with the other. This is often referred to as feature mapping.

Furthermore, there are two additional layers besides convolution: pooling and flattening. The pooling method moves a window across an image's pixels, fusing local features into a single feature. Pooling can take a variety of forms:

1. Max pooling: when the pixel with the highest intensity is chosen over all other pixels in the window.
2. Average pooling: where a window returns the average of all the pixels.

Pooling is primarily used to reduce the size of data. At this stage, the model retains significant information about the local pixels, such as the highest intensity or the average of all pixels, and discards other irrelevant data.

Following extraction of features from images via CNNs and pooling, these features should be fed to ANNs for classification. The flattened layer converts these 2D features to 1D features, which are then connected to ANNs for further processing. The basic architecture of a CNN based on artificial neural networks is depicted in Fig. 9.10.

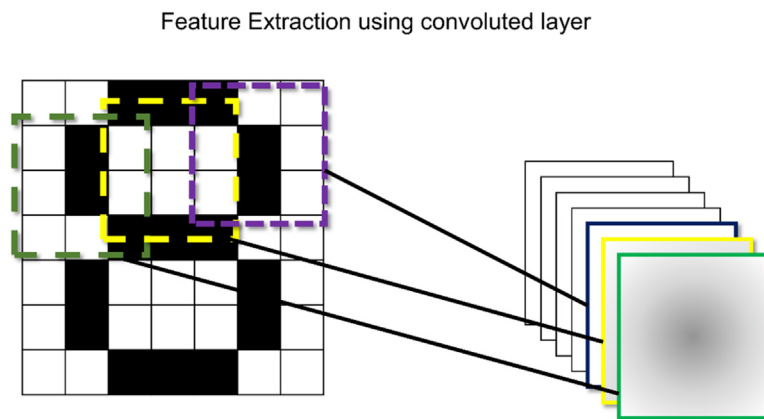
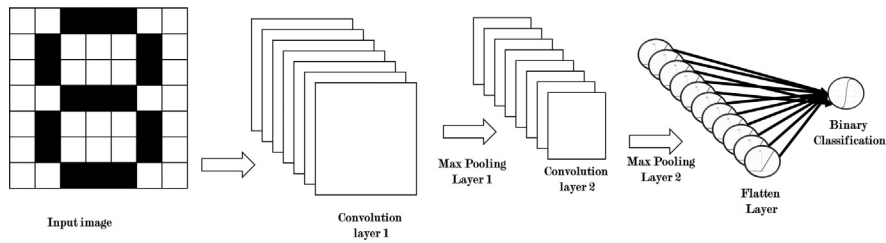


FIG. 9.9

Feature extraction using CNNs.

**FIG. 9.10**

Architecture of a CNN bases neural system.

9.2.10 Unsupervised learning

Unsupervised learning differs from supervised learning in that the data that we send to a computer is not labeled. To put it another way, we're not feeding the machine by hand; instead, we're making it smart enough to interpret the data we're sending it on its own. Data points with corresponding features are fed to unsupervised learning algorithms. The algorithm takes the features of the data and extracts useful bits of information based on similarities and differences within the data before trying to deduce trends.

Clustering and discovery of outliers are two extensive applications of unsupervised learning. Clustering is the most popular method of unsupervised machine learning. Clustering is the act of sorting the data into many groups. Clusters are the names of the groups segregated by machines and researchers then focus on the clusters to decipher the segregation. For example, in gene expression analysis, clustering is used to segregate the genes according to their expression patterns. Once an unsupervised algorithm group them, bioinformaticians try to understand their cumulative effects. The computer looks for data points with identical parameters in the whole data set. After that, based on each of these parameters, a data point is assigned to a special cluster.

Outlier detection, on the other hand, recognizes something that deviates from a general pattern. If you've been seeing white cars on the road for a while, seeing a red car will attract your attention. The work of the outlier detection systems is carried out in this way. This technique can be used to diagnose diseases and variations, human data entry errors, and much more.

9.2.11 K-means clustering

We've seen examples of supervised learning in the above section so far, where labels have been used, and classifiers are built using features and labels. Unsupervised learning, on the other hand, does not have the labeled data (Altman and Krzywinski, 2017). They find hidden patterns in datasets, build clusters based on dataset similarity, and group data points that are more close to each other (called a cluster). K-Means clustering is a popular unsupervised learning algorithm, it uses the simple

iterative rule to separate unlabeled datasets to form clusters. In biology, clustering has a wide range of applications, including analysis of expression results, drug repurposing, and categorization of organisms or proteins, among others.

The first step in K-means clustering is to tell the algorithms how many clusters or groups a user anticipates, i.e., the value of “K”. The algorithm will cluster the data into “K” number of groups. Selecting the value of K requires knowledge of the domain or simply one can iterate the clustering process with various number of Ks and find the most suitable one based on user define parameters.

After determining the value of K, they are referred to as centroids and are randomly located within the dataset space. The sum of the squared distance between the data points and the centroid cluster (arithmetic mean of all data points belonging to that cluster) is the minimum, such that the data points closer to those randomly scattered centroids are assigned to the cluster.

In the third step, the centroids are moved to the cluster center after assigning points closer to the k clusters, as seen in [Fig. 9.11](#). The mean of all data points in the cluster is the cluster nucleus. Closer data points are assigned to k clusters after the centroids are moved to the cluster mean, and the steps are repeated until the centroids stop moving or the movement is very slow.

Finally, we acquire well-defined k clusters from which we observe similar data points allocated to clusters to discover secret pathways and derive valuable knowledge from the dataset. Calculation of the coordinates or properties of the data points generates the nucleus of the cluster.

9.2.12 Reinforcement learning

Reinforcement learning is the process of teaching machine learning models to make a variety of decisions. The agent learns to accomplish a goal in a static, potentially tricky environment. Machine learning algorithms face a game-like scenario in reinforcement learning to solve the problem, the algorithms employ trial and error methods. Getting the ML algorithm trained is either rewarded or punished, which means it gains or loses points for the hits it makes. Its goal is to maximize the total reward or scores ([Collins and Cockburn, 2020](#)).

The idea that the user creates a layout of the game or the environment—that is, the game rules—provides the model with no tips or suggestions on how to solve the game. Starting with fully randomized trials and progressing to more sophisticated methods, it is up to the model to determine the steps to optimize the rewards. Reinforcement learning, which employs rewards and punishments through multiple testing, is possibly the most effective method for teaching an algorithm creativity. Unlike humans, artificial intelligence can derive information from thousands of concurrent gameplays if a reinforcement learning algorithm is allowed to run on a robust computing infrastructure.

Reinforcement Learning is a subset of machine learning that teaches an agent how to choose an action from an action space in each context to maximize rewards over time. Reinforcement Learning has four components:

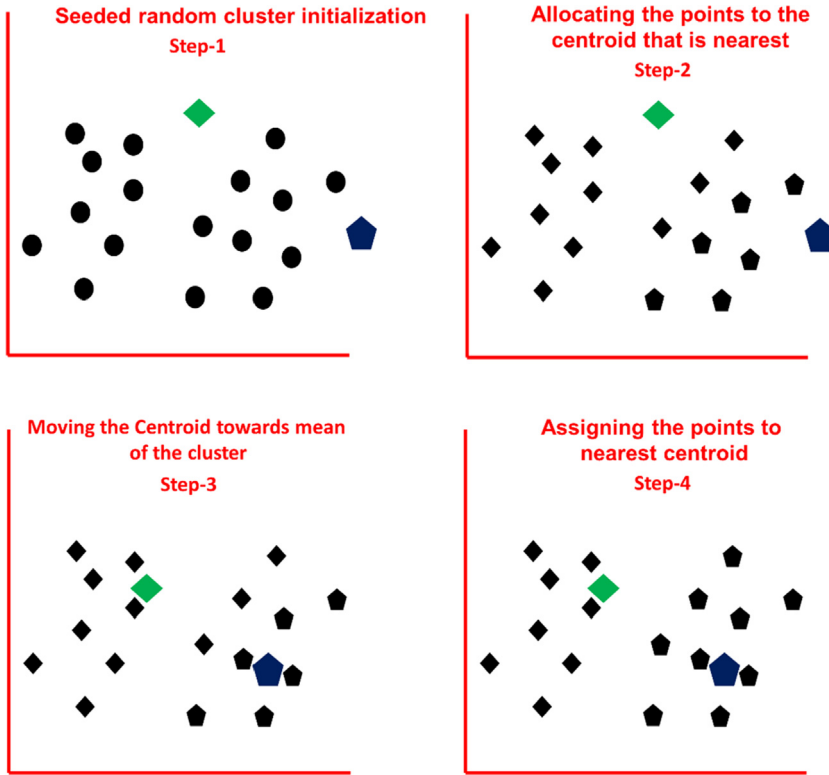


FIG. 9.11

Steps of K means clustering.

1. Agent: The ML model
2. Environment: The physical or virtual environment in which the agent operates.
3. Alternatives or steps: The agent's action that causes a change in the state of the environment.
4. Choice-based reward system: The evaluation of a particular action, which can be positive or negative.

The difference between supervised, unsupervised, and reinforcement learning is their objective and how they are achieving it. Both supervised and unsupervised learning aims to discover and learn patterns that produce relatively static results. On the other hand, RL's goal is to have a plan that tells the agent what action to take at each stage, which complicates the operation. Reinforcement Learning does not explicitly provide the correct answer; instead, the agent must learn the answer through trial and error. The agent's only metric is the reward received after the procedure is completed, indicating whether it is progressing.

In contrast, during training in supervised learning, the algorithms are given correct answers. A Reinforcement Learning agent must strike the right balance between discovering the correct steps by searching for new sources of rewards and using previously identified reward sources. Unsupervised and supervised learning systems, on the other hand, derive the solution directly from training data without considering alternative solutions. Reinforcement Learning is a multi-decision mechanism that generates a decision-making sequence based on the amount of time required to complete a task. In contrast, supervised learning is a one-decision process: one event, one prediction.

9.3 Evaluation of machine learning models

Once models have been trained, it is critical to assess their outputs or see how well they can perform (Antoniou and Mamdani, 2021). A test set is used to evaluate the performance of the models better. Typically, the entire data set is divided into two parts: 80% is used to train the machine and 20% is used to test how well the model predicts. The model is not shown the test data during training. After the algorithm has been trained, the previously unseen data is used to assess how well the algorithm performs in new data. The algorithm predicts the test data outputs, which are then compared to the original labels. The predicted and actual values are compared in a confusion matrix to measure how well the model performed (Fig. 9.12).

Various mathematical metrics can evaluate a model using the confusion matrix values, such as accuracy, precision, and recall.

9.3.1 Accuracy

The number of correctly predicted values divided by the total number of instances in the test set is the rate of accurate prediction.

Confusion Matrix		
	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

FIG. 9.12
Confusion matrix.

$$TP + TN / TP + FP + TN + FN = \text{accuracy}$$

The ratio of true positive to the total number of instances considered positive by the model is known as precision.

The true positive rate, also known as the sensitivity of the formula, is called recollection. i.e., the total number of positive instances in the dataset of the survey divided by the number of true positive instances.

$$TP / TP + FN = \text{ranking of R F1}$$

Precision and recall are often specified a single numerical value known as the F1 score. The harmonic mean of precision and recall.

$$2 * ((\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})) \text{ F1 Score F1}$$

9.3.2 Receiver Operating Characteristic (ROC) Curvature

The Receiver Operating Characteristic (ROC) Curve is another method for testing classifiers. It is a graph showing the difference between the true positive rate (TPR), also known as sensitivity, and the false positive rate (FPR) (1 - Specificity).

A preferred curve shows the increase in the true positive rate vs. the false positive rate. The more area there is under the curve, the more accurate the model is (Fig. 9.13).

The parameters for evaluating the regression model are very different. Since regression operates with a continuous data set, it requires sophisticated metrics to test it. Variance, mean squared error, and R squared coefficient are typical parameters for assessing regression models.

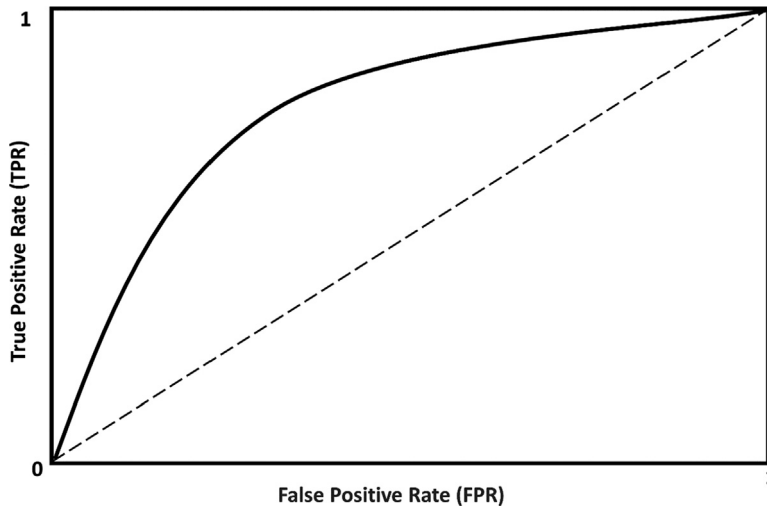


FIG. 9.13

Curve of the ROC.

9.3.3 Cross-validation

Cross-validation is a term used to describe the method of comparing two or more subjects.

As discussed above, when performing a machine learning project, we divide our dataset into two sets at random, training and test sets. This approach has a drawback: if we lose about 20% of our data for testing our model, it may not be able to train properly for real-world challenges. Some vital information will be skipped as we split the data set. When the data collection is small, the problem gets worse. To solve this problem, we use a technique called cross-validation. We divide the dataset into subsets and then we train and evaluate our model using various combinations of these subsets (Fig. 9.14). Two of these methods are K-fold validation and Leave One Out Cross Validation (LOOCV).

9.3.4 Testing and validating

When a large amount of data is available, it can be divided into three parts: training, validation, and testing sets. While training makes up the majority of the part from rests and is useful for teaching the model. The validation set is kept separate from the training data and is used for evaluating the trained model for fine-tuning its parameters, then train and evaluate again. This cycle is repeated until a model with consistent accuracy is discovered through permutations and combinations of various model parameters. The test set is used for the final evaluation because it is still hidden from the model and provides an unbiased evaluation result. Testing and

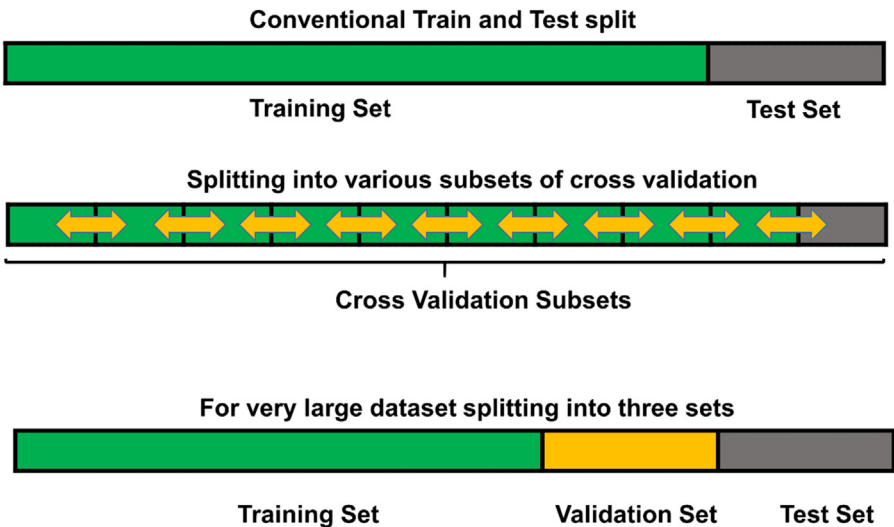


FIG. 9.14

Multiple configurations for training, validating, and testing of models.

validation on the same set of data is not recommended because, with each cycle, the model becomes biased toward the validation set due to favored parameters.

9.4 Optimization of models

Model optimization is a key step in improving the accuracy of the predicted outcomes. Based on the complexity of an ML algorithm, we can have various hyperparameters as we have discussed in the above sections. The purpose of fine-tuning a machine learning algorithm is to choose the best model and parameter values. In general, models are iteratively configured and validated using a number of parameter combinations. Continuous training and validation ensure that a model is optimized and performs well. Here we have discussed a few methods of optimization of ML models.

9.4.1 Parameter searching

There are two widely used methods for optimizing models by searching parameter:

Grid search is a method for finding the best hyperparameter combination for a model by scanning the space of all hyperparameter combinations. Depending on the type of model used, certain hyperparameters are required. Grid search is a technique for determining the optimal hyperparameters for any machine learning model. It should be noted that grid searching is computationally intensive and can take a long time to optimize a model. It iterates through all possible hyperparameter combinations and saves a model for each one. This method can be made more effective with domain knowledge by providing a limited set of parameters to evaluate.

[Bergstra and Bengio \(2012\)](#) proposed the concept of a system's hyperparameters being randomly searched. This is a completely different type of search than the grid approach. Rather than combing through all possible hyperparameter combinations, a randomized search selects a few sample points from the distribution and performs the calculations on those points.

9.4.2 Ensemble methods

Two heads are better than one means that when two people work together to solve a problem, they are more likely to succeed than when one person works alone. Ensemble methods are a class of techniques that include the development of several models and their subsequent combination to achieve improved results. Generally, ensemble models yield more precise solutions than a single model does. Voting and averaging are two of the most straightforward ensemble techniques. They are both simple to comprehend and simple to execute. Classification is accomplished by voting, while regression is performed by averaging. Both methods begin by developing multiple classification/regression models on a training dataset. Each base model can be constructed using different splits of the same training dataset

and the same algorithm, or by combining the same dataset and different algorithms, or by any other approach.

9.5 Main challenges of machine learning

Machine learning enables us to make more informed, data-driven decisions more quickly than traditional methods allow. However, machine learning, like any other system, has its own set of challenges. While machine learning applications in biology are advancing rapidly, technology as a whole has a long way to go. When it comes to developing a machine learning solution, there are numerous obstacles and issues to overcome. With this in mind, let us examine some of the challenges faced by applied machine learning projects.

9.5.1 Insufficient quantity of training data

Having sufficient data to train the algorithm is critical to the machine learning project's success. Many machine learning programs may fail to approximate the real-world scenario due to a lack of data. As a result, data collection is the most critical component of any machine learning project. The amount of data depends on the complexity of the problem and the chosen ML algorithm. While high throughput technologies have significantly increased the amount of data available for biological studies, the integration of divergent types of data remains a limitation for big data in biology. High-throughput data from transcriptomics, proteomics, metabolomics, and genomics must be integrated into a meaningful large dataset. It remains a challenge that, despite the production of astronomical amounts of data in the biological domain, the overall mechanism of a system such as phenotypes remains difficult to comprehend due to a lack of data integration.

9.5.2 Non-representative training data

Training data must be a representative sample of the population. A sample data is a subset of the population's data points used to train a model. The sample must accurately reflect the population. To generalize the output of a machine learning model, it must be trained on well-represented data; otherwise, it will encounter a blind spot for predictions on data with few representations.

Sampling noise, also known as non-representative data, occurs when the sample size is insufficient, but even large samples can be non-representative if the sampling procedure is inaccurate. This is frequently referred to as sampling biased. It is critical to note that when sampling bias is reduced, the variance increases, preventing the model from generalizing to unseen data; when variance is reduced, the bias increases. This phenomenon also referred to as the bias-variance trade-off; one should balance these two parameters.

9.5.3 Quality of data

Machine learning algorithms pose a huge challenge when it comes to data consistency. These systems require high-quality data to prevent over or under estimation during training and testing cycles. As a result, the consistency of the data used in any machine learning system has a huge impact on its development. Consequently, even small variations in the training data can result in major alteration in the algorithm's output. Complex and dynamic problems require not only large quantities of data but also diverse and informative data. While curating data sets, the missing, misinterpreted, and inaccurate values may affect data quality.

9.5.4 Irrelevant features

The independent variables in any machine learning problem are the features. The output, which is a dependent variable, is predicted using these independent variables. During training, the majority of machine learning algorithms derive an approximate function from these features. As a result, it is critical to select relevant features for inclusion in a machine learning algorithm. Relevant features can be used to direct the algorithm toward a more desirable output. Inappropriate features mislead an algorithm, increase the size of the data, and complicate the problem unnecessarily. In many cases, feature redundancy can also result in an increase in data size. Two or more highly correlated features have little “value” in terms of training because the presence of one can always (or almost always) be used to determine the presence of the other. If this is the case, there is no reason to include both features, as their combined effect on the predictions will be negligible.

Sometimes the simplest solution is the best. To be more specific, we can use dimensionality reduction while retaining variance, as in PCA, by using the principal components as features. Furthermore, because more features introduce more noise, we can use a variety of methodologies and techniques to select a subset of the feature space to aid our models in performing better.

We have discussed the difficulties and limitations of machine learning as a result of the scarcity of useful data thus far. Data has become the new gold as a result of advancements in machine learning. But there are some limitations in algorithms themselves, which we will discuss in the next section.

9.5.5 Overfitting or underfitting on training data

Overfitting occurs when a model performs exceptionally well during training but performs poorly on test data. It takes place when an algorithm acquires an excessive amount of knowledge about the training data. When a model acquires unnecessary extra details and noise from training data, it has a negative impact on the model's performance on test data. As a result, the model will be incapable of generalizing the real-world situation. Generally, nonlinear machine learning algorithms with a large number of parameters overfit the training data. Overfitting can be avoided by fine-tuning the parameters.

When a system cannot precisely capture the relationships between features and output variables, this is referred to as underfitting. This impairs the algorithm's ability to decode the data's underlying pattern. Underfitting is a good indication that the algorithm is unsuitable for the given dataset. Underfitting is typical in situations where there is insufficient data to train the algorithm. Fewer data points result in an inconsistency in the model's training, which then fails to perform well on the test dataset. Increasing the amount of data available to the algorithm may be one way to ensure that it has enough information to identify general trends and patterns. Another way to tackle underfitting is to look for other algorithms.

References

- Altman, N., Krzywinski, M., 2017. Points of significance: Clustering. *Nat. Methods* 14, 545–546. <https://doi.org/10.1038/NMETH.4299>.
- Altman, N., Krzywinski, M., 2015. Points of significance: Simple linear regression. *Nat. Methods* 12, 999–1000. <https://doi.org/10.1038/NMETH.3627>.
- Antoniou, T., Mamdani, M., 2021. Evaluation of machine learning solutions in medicine. *CMAJ (Can. Med. Assoc. J.)* 193, E1425–E1429. <https://doi.org/10.1503/CMAJ.210036/TAB-RELATED-CONTENT>.
- Bergstra, J., Bengio, Y., 2012. Random search for hyper-parameter optimization. *JMLR* 13 (10), 281–305.
- Collins, A.G.E., Cockburn, J., 2020. Beyond dichotomies in reinforcement learning. *Nat. Rev. Neurosci.* 21 (10), 576–586. <https://doi.org/10.1038/s41583-020-0355-6>.
- Greener, J.G., Kandathil, S.M., Moffat, L., Jones, D.T., 2021. A guide to machine learning for biologists. *Nat. Rev. Mol. Cell Biol.* 23 (1), 40–55. <https://doi.org/10.1038/s41580-021-00407-0>.
- Kotsiantis, S.B., 2011. Decision trees: A recent overview. *Artif. Intell. Rev.* 39 (4), 261–283. <https://doi.org/10.1007/S10462-011-9272-4>.
- Larrañaga, P., Calvo, B., Santana, R., Bielza, C., Galdiano, J., Inza, I., Lozano, J.A., Armañanzas, R., Santafé, G., Pérez, A., Robles, V., 2006. Machine learning in bioinformatics. *Briefings Bioinf.* 7, 86–112. <https://doi.org/10.1093/BIB/BBK007>.
- Lecun, Y., Bengio, Y., Hinton, G., 2015. Deep learning. *Nature* 521 (7553), 436–444. <https://doi.org/10.1038/nature14539>.
- Lever, J., Krzywinski, M., Altman, N., 2016. Points of significance: Logistic regression. *Nat. Methods* 13, 541–542. <https://doi.org/10.1038/NMETH.3904>.
- Parry, R.M., Jones, W., Stokes, T.H., Phan, J.H., Moffitt, R.A., Fang, H., Shi, L., Oberthuer, A., Fischer, M., Tong, W., Wang, M.D., 2010. k-Nearest neighbor models for microarray gene expression analysis and clinical outcome prediction. *Pharmacogenom. J.* 10 (4), 292–309. <https://doi.org/10.1038/tpj.2010.56>.
- Wang, P.W., Lin, C.J., 2014. Support vector machines. In: *Data Classification: Algorithms and Applications*. <https://doi.org/10.1201/b17320>.